```
CREATE TABLE employees_sub (
    emp_id INT PRIMARY KEY,
    name VARCHAR(50),
    department_id VARCHAR(10),
    salary INT
);

CREATE TABLE departments_sub (
    department_id VARCHAR(10) PRIMARY KEY,
    department_name VARCHAR(50),
    location VARCHAR(50)
);

CREATE TABLE sales_sub (
    sale_id INT PRIMARY KEY,
    emp_id INT,
    sale_amount INT,
    sale_date DATE
);

INSERT INTO employees_sub VALUES (101, 'Abhishek', 'D01', 62000), (102, 'Shubham', 'D01', 58000), (103, 'Priya', 'D02', 67000), (104, 'Rohit', 'D02', 64000), (105, 'Neha', 'D03', 72000), (106, 'Aman', 'D03', 55000), (107, 'Ravi', 'D04', 60000), (108, 'Sneha', 'D04', 75000), (109, 'Kiran', 'D05', 70000), (110, 'Tanuja', 'D05', 65000);
INSERT INTO departments_sub VALUES ('D01', 'Sales', 'Mumbai'), ('D02', 'Marketing', 'Delhi'), ('D03', 'Finance', 'Pune'), ('D04', 'HR', 'Bengaluru'), ('D05', 'IT', 'Hyderabad');
INSERT INTO sales_sub VALUES (201, 101, 4500, '2025-01-05'), (202, 102, 7800, '2025-01-10'), (203, 103, 6700, '2025-01-14'), (204, 104, 12000, '2025-01-20'), (205, 105, 9800, '2025-02-02'), (206, 106, 10500, '2025-02-05'), (207, 107, 3200, '2025-02-09'), (208, 108, 5100, '2025-02-15'), (209, 109, 3900, '2025-02-20'), (210, 110, 7200, '2025-03-01');
```

1. Retrieve the names of employees who earn more than the average salary of all employees.

➔ Snippet to run:

```
SELECT name
FROM employees_sub
WHERE salary > (SELECT AVG(salary) FROM employees_sub);
```

Result Grid | Filter Rows:

name
Priya
Neha
Sneha
Kiran
Tanuja

The Subquery: `(SELECT AVG(salary) FROM employees_sub)` calculates the mean salary of everyone in the table.
The Outer Query: Compares each employee's individual salary against that calculated average.
Logic: If the average is 64,800, the query only displays names like Priya, Neha, Sneha, and Kiran.

2. Find the employees who belong to the department with the highest average salary.

➔ Snippet to run:

```
SELECT name, department_id
FROM employees_sub
WHERE department_id = (
   SELECT department_id
   FROM employees_sub
   GROUP BY department_id
   ORDER BY AVG(salary) DESC
   LIMIT 1
);
```

Result Grid | Filter Rows:

name	department_id
Ravi	D04
Sneha	D04

The Subquery: Groups employees by department_id, calculates the AVG(salary) for each, sorts them from highest to lowest, and picks the top one (D05 in this dataset).
The Outer Query: Filters the employees_sub table to only show people who work in that specific high-paying department.
Logical Flow: Since Department D05 (IT) has an average salary of 67,500 ($(70,000 + 65,000) / 2$), the outer query pulls the names Kiran and Tanuja

3. List all employees who have made at least one sale.

➔ Snippet to run:

```
SELECT name
FROM employees_sub
WHERE emp_id IN (SELECT DISTINCT emp_id FROM sales_sub);
```

Result Grid

name
Abhishek
Shubham
Priya
Rohit
Neha
Aman
Ravi
Sneha
Kiran
Tanuja

The Subquery: Extracts a unique list of all `emp_id`s present in the `sales_sub` table.
The Outer Query: Checks the `employees_sub` table and returns the names of anyone whose ID appears in that list.
Result: Since every employee in our dataset (101 through 110) has at least one entry in the Sales table, all names will be returned.

4. Find the employee with the highest sale amount.

➔ Snippet to run:

```
SELECT name
FROM employees_sub
WHERE emp_id = (
   SELECT emp_id
   FROM sales_sub
   WHERE sale_amount = (SELECT MAX(sale_amount) FROM sales_sub)
);
```

Result Grid

name
Rohit

The Deepest Subquery: `(SELECT MAX(sale_amount) FROM sales_sub)` identifies the highest individual transaction, which is **12,000**.
The Middle Subquery: Finds the `emp_id` (104) associated with that 12,000 sale.
The Outer Query: Looks up the name for `emp_id` 104 in the `employees_sub` table.

5. Retrieve the names of employees whose salaries are higher than Shubham's salary.

➔ Snippet to run:
SELECT name
FROM employees_sub
WHERE salary > (SELECT salary FROM employees_sub WHERE name = 'Shubham');

name
Abhishek
Priya
Rohit
Neha
Ravi
Sneha
Kiran
Tanuja

The Subquery: `(SELECT salary FROM employees_sub WHERE name = 'Shubham')` returns the value **58,000**.
The Outer Query: Filters the table for anyone earning strictly more than 58,000.
Logical Result: Since Abhishek earns 62,000 and Priya earns 67,000, they (along with several others) will appear in the results.

6. Find employees who work in the same department as Abhishek.

➔ Snippet to run:
SELECT name
FROM employees_sub
WHERE department_id = (SELECT department_id FROM employees_sub WHERE name = 'Abhishek')

AND name != 'Abhishek';

Result Grid

	name
▶	Shubham

The Subquery: `(SELECT department_id FROM employees_sub WHERE name = 'Abhishek')` identifies that Abhishek is in department **D01**.
The Outer Query: Searches for all employees in **D01**.
The Filter: `AND name != 'Abhishek'` is added to exclude Abhishek himself from the final list of his coworkers.

7. List departments that have at least one employee earning more than ₹60,000.

➔ Snippet to run:
SELECT department_name
FROM departments_sub
WHERE department_id IN (SELECT DISTINCT department_id FROM employees_sub
WHERE salary > 60000);

	department_name
▶	Sales
	Marketing
	Finance
	HR
	IT

The Subquery: Identifies every `department_id` where at least one person makes > 60,000 (D01, D02, D03, D04, D05).
The Outer Query: Matches those IDs to the `departments_sub` table to get the human-readable names.
Logical Result: Since every department in our sample data has at least one high earner, all department names will be returned.

8. Find the department name of the employee who made the highest sale.

➔ Snippet to run:
SELECT department_name
FROM departments_sub
WHERE department_id = (
  SELECT department_id

```
    FROM employees_sub
    WHERE emp_id = (
        SELECT emp_id
        FROM sales_sub
        ORDER BY sale_amount DESC
        LIMIT 1
    )
);
```

Result Grid

	department_name
▶	Marketing

The Innermost Subquery: `(SELECT emp_id FROM sales_sub ORDER BY sale_amount DESC LIMIT 1)` identifies the employee who made the largest sale (emp_id 104).
The Middle Subquery: Finds the `department_id` associated with that employee (D02).
The Outer Query: Retrieves the actual `department_name` (Marketing) for that ID.

9. Retrieve employees who have made sales greater than the average sale amount.

➔ Snippet to run:

```
SELECT DISTINCT name
FROM employees_sub
WHERE emp_id IN (
    SELECT emp_id
    FROM sales_sub
    WHERE sale_amount > (SELECT AVG(sale_amount) FROM sales_sub)
);
```

Result Grid

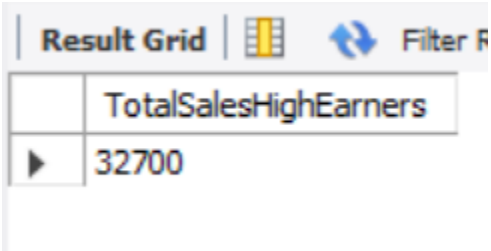
	name
▶	Shubham
	Rohit
	Neha
	Aman
	Tanuja

The Deepest Subquery: Calculates the average of all sales amounts in the `sales_sub` table.
The Middle Subquery: Filters for all `emp_id`s who have at least one transaction above that average.
The Outer Query: Returns the names of those employees from the `employees_sub` table.

10. Find the total sales made by employees who earn more than the average salary.

➔ Snippet to run:

```
SELECT SUM(sale_amount) AS TotalSalesHighEarners
FROM sales_sub
WHERE emp_id IN (
   SELECT emp_id
   FROM employees_sub
   WHERE salary > (SELECT AVG(salary) FROM employees_sub)
);
```



	TotalSalesHighEarners
▶	32700

The Deepest Subquery: `(SELECT AVG(salary) FROM employees_sub)` calculates the average salary of all employees (which is approximately ₹64,300).
The Middle Subquery: Identifies the `emp_id` for everyone earning above that average (Priya, Neha, Sneha, Kiran, and Tanuja).
The Outer Query: Sums up the `sale_amount` from the `sales_sub` table for only those specific IDs.

11. Find employees who have not made any sales.

➔ Snippet to run:

```
SELECT name
FROM employees_sub
WHERE emp_id NOT IN (SELECT DISTINCT emp_id FROM sales_sub);
```

Result Grid

	name

The Subquery: Creates a list of every `emp_id` that exists in the `sales_sub` table.
The Outer Query: Looks for names in the `employees_sub` table whose `emp_id` is **not** present in that list.
Logic: In the current dataset provided in the document, every employee (101-110) has made a sale, so this query would return an empty set. However, in a real-world database, this is how you find underperforming staff.

12. List departments where the average salary is above ₹55,000.

➔ Snippet to run:

```
SELECT department_name
FROM departments_sub
WHERE department_id IN (
   SELECT department_id
   FROM employees_sub
   GROUP BY department_id
   HAVING AVG(salary) > 55000
);
```

Result Grid Filter Rows

	department_name
▶	Sales
	Marketing
	Finance
	HR
	IT

The Subquery: It groups the employees by their department_id and calculates the AVG(salary) for each group. The HAVING clause then filters out any department where the average salary is ₹55,000 or less.
The Outer Query: It takes the list of department_ids that passed the filter and retrieves their corresponding department_name from the departments_sub table.

Data Insight: Since the lowest average salary in our dataset is ₹58,500 (Department D03: (72,000 + 55,000) / 2 = 63,500$; Department D01: (62,000 + 58,000) / 2 = 60,000, all departments will be listed.

13. Retrieve department names where the total sales exceed ₹10,000.

➔ Snippet to run:

```
SELECT department_name
FROM departments_sub
WHERE department_id IN (
    SELECT department_id
    FROM employees_sub
    WHERE emp_id IN (
        SELECT emp_id
        FROM sales_sub
        GROUP BY emp_id
        HAVING SUM(sale_amount) > 10000
    )
);
```

	department_name
▶	Marketing
	Finance

The Innermost Subquery: Aggregates the `sale_amount` for each employee in the `sales_sub` table and filters for those with a total greater than ₹10,000.
The Middle Subquery: Finds the `department_id` for the high-performing employees identified in the first step.
The Outer Query: Looks up the human-readable `department_name.`
Key Matches: Rohit (emp_id 104) and Aman (emp_id 106) both exceed this threshold, leading to the "Marketing" and "Finance" departments being returned.

14. Find the employee who has made the second-highest sale.

➔ Snippet to run:

```
SELECT name
FROM employees_sub
WHERE emp_id = (
    SELECT emp_id
    FROM sales_sub
    WHERE sale_amount = (
        SELECT MAX(sale_amount)
```

```
        FROM sales_sub
        WHERE sale_amount < (SELECT MAX(sale_amount) FROM sales_sub)
    )
);
```

	name
▶	Aman

The Innermost Subquery: `(SELECT MAX(sale_amount) FROM sales_sub)` finds the absolute highest sale, which is **12,000**.
The Second Subquery: Finds the `MAX(sale_amount)` that is *less than* 12,000, which identifies **10,500** as the second-highest amount.
The Middle Subquery: Retrieves the `emp_id` (106) associated with that 10,500 sale.
The Outer Query: Looks up the name for `emp_id` 106, which is **Aman**.

15. Retrieve the names of employees whose salary is greater than the highest sale amount recorded.

➔ Snippet to run:

```
SELECT name
FROM employees_sub
WHERE salary > (SELECT MAX(sale_amount) FROM sales_sub);
```

	name
▶	Abhishek
	Shubham
	Priya
	Rohit
	Neha
	Aman
	Ravi
	Sneha
	Kiran
	Tanuja

The Subquery: `(SELECT MAX(sale_amount) FROM sales_sub)` returns the highest sale recorded in the system, which is **12,000**.

The Outer Query: Filters the `employees_sub` table for anyone whose salary is greater than 12,000.
Result: Since all employees in our dataset have salaries ranging from 55,000 to 75,000, **everyone** in the table will be returned in this specific result set.